

1 Assignment overview

This assignment will involve you designing, building, evaluating and critiquing systems for two applied machine learning tasks.

1. Task 1: Natural Language Processing (50%): A system for performing sentiment analysis (classifying positive or negative reviews).
2. Task 2: Computer Vision (50%): A system for performing face alignment (locating facial landmarks in images of people).

This assignment is worth 100% of the grade for this module. It is designed to ensure you can demonstrate achieving the learning outcomes for this module, which are to:

- Determine the applicability of different machine learning models to data found in real-world applications.
- Propose designs for simple systems, including appropriate pre-processing, to solve practical problems using machine learning.
- Implement and document computer programmes that learn and apply machine learning models to realistic data.
- Critically evaluate the efficacy of proposed systems and appropriately communicate this analysis.

The main skills that this assignment tests are your understanding, critical thinking and originality of thought. Most of the marks are awarded for your ability to demonstrate these skills. There are also marks available for your ability to obtain solutions to the tasks, i.e., working code to successfully perform sentiment analysis and face alignment.

2 What to hand in?

1. A report that comprises a *maximum* of 3000 words, including figure captions but excluding references. The report should be written in two sections, one for each task. For each task, you should cover the following points. More detail is provided in Sections 3 and 4 below.
 - A summary and justification for all the steps in your system, including preprocessing, choice of features (more generally, this is called the “representation”, particularly in deep-learning settings), and prediction models. Explaining the system diagrammatically is expected. Include references to appropriate sources, e.g., literature, software, technical reports.
 - Results of your experiments. This should include some discussion of quantitative (number based) and qualitative (example based) comparisons between different approaches that you have experimented with.
 - Examples of failure cases in your system and a critical analysis of these, identifying potential biases.

2. Either .ipynb files or .py files containing annotated code for all data preprocessing, model training and validation. You can also include a link in your report to a public [GitHub](#) repository that contains your code (using GitHub is standard industry and academic practice).
3. For Task 1: A csv file that contains the predicted labels on the **test** set of text, found in the csv file accessible from the corresponding link in Table 1. You **must** use the provided “save_as_csv” function in the Google Colab worksheet (see Table 1 for the link) to process an array of shape (number_test_data, 1) to a csv file. Please make sure you run this on the right data and submit in the correct format to avoid losing marks.
4. For Task 2: A csv file that contains the face landmark positions on the **test** set of images, found in the compressed numpy file accessible from the corresponding link in Table 2. You **must** use the provided “save_as_csv” function in the Google Colab worksheet (see Table 2 for the link) to process an array of shape (number_test_image, number_points, 2) to a csv file. Please make sure you run this on the right data and submit in the correct format to avoid losing marks.

Please only use the .zip archive format for your submission and do not include the original datasets (we already have them!).

Note of caution! Do not reorder the test data or your predictions will not match up with our test labels / points!

3 Task 1: Sentiment analysis

You will design a system to detect positive or negative sentiment in movie reviews. However, there is a twist - there is spam mixed into the provided datasets that has been randomly and evenly distributed between the two classes, such that you don't have access to the “true” label for the spam data. Therefore to prevent an unwanted drop in performance of your system, you will need to consider alternative techniques to directly applying supervised classifiers out-of-the-box. Note that the spam is not from movie reviews (it is classic email-type spam).

3.1 Mark allocation

15 Marks **Description of methods**

Justify and explain design decisions for the sentiment analysis system. You should state clearly:

- Any text pre-processing steps you have used, and why.
- What text features/representations you have used (for example, word embeddings). Describe how they were computed, and why you chose them.
- What predictions methods you have used; what machine learning task this corresponds to; the type of model(s) that you have used; and the loss function that each model is trained with.
- Design and hyperparameter choices should be explained and justified.
- Compute usage: what hardware you used and how long it took to train your model(s).

For top marks, you should consider more than one approach. Using diagrams and/or flowcharts is expected as part of your description of a) your pre-processing and feature/representation pipeline; and b) your model structure/architecture.

15 Marks **Analysis of results**

You should include quantitative (number based) and qualitative (example based) evaluations of different approaches that you have tried (on the held-out validation set).

- Quantitative evaluation includes at least computing the confusion matrix of your predictions.
- Qualitative evaluation includes at least subjective interpretation of language.
- Explicitly define any evaluation metrics and ensure they are appropriate for the task.
- Note that we are not only interested in your final prediction results, but also your explanation of why they performed well or badly. For example, you should identify failure cases and systematic bias, and provide plausible explanations for why they occur.

10 Marks **Performance on external dataset**

Choose one trained model and evaluate its performance on the NLTK corpus “movie reviews” dataset. You can access it directly from NLTK in your code with:

```
nltk.corpus import movie_reviews
```

You should provide explanations for your model’s generalisation performance on this dataset. Note that there is no spam in this dataset!

10 Marks **Accuracy of sentiment analysis**

These marks are allocated based on the performance of one trained model of your choice. This will be evaluated on the held out test data. The test data (without labels) are provided in the csv file accessible from the corresponding link in Table 1. The error on the predicted labels will be calculated after submission. Marks will be awarded for accuracy, based on the confusion matrix of your predictions. Note that there is spam in the test data, so your model should assign a dummy label (i.e., not 0 or 1) to these samples.

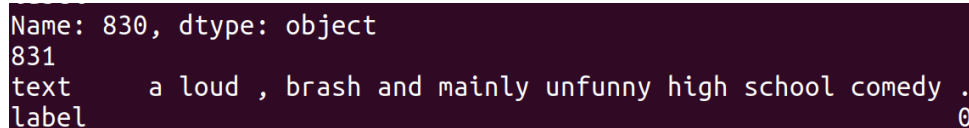
3.2 Task 1 most important links

Contents	File name	Link
Training text and label data	sentiment_analysis_training_data.csv	link1
Validation text and label data	sentiment_analysis_validation_data.csv	link2
Test text data (without labels)	sentiment_analysis_test_data.csv	link3
Colab worksheet with some useful functions	aml_task1_nlp_worksheet.ipynb	link4

Table 1: Task 1 important links.

3.3 Where to start?

Sentiment analysis is covered in week 2, and the accompanying lab session in week 3. Word embeddings are covered in week 5, lecture 9. Various types of word embeddings are readily available, using e.g., [NLTK](#). We have included a basic Colab worksheet (see link in Table 1) illustrating how to load the data and print random text examples based on their labels. An example print-out is shown in Figure 1.



```
Name: 830, dtype: object
831
text      a loud , brash and mainly unfunny high school comedy .
label      0
```

Figure 1: Example of true negative sentiment in the training dataset.

4 Task 2: Face alignment

You will design a system to perform face alignment on images of people's faces. But again there is a twist - there is variability in the dataset, such as transformations that distort the images. Therefore to prevent an unwanted drop in performance in your system, you will need to consider techniques that provide robustness to sources of unwanted variability in your system.

4.1 Mark allocation

15 Marks **Description of methods**

Justify and explain design decisions for the face alignment system. You should state clearly:

- Any image pre-processing steps you have used, and why.
- What image features/representation you have used (for example, feature descriptors), describe how they were calculated, and why you chose them.
- What predictions methods you have used; what machine learning task this corresponds to; the type of model(s) that you have used; and the loss function that each model is trained with.
- Design and hyperparameter choices should be explained and justified.
- Compute usage: what hardware you used and how long it took to train your model(s).

For top marks, you should consider more than one approach. Using diagrams and/or flowcharts is expected as part of your description of a) your pre-processing and feature/representation pipeline; and b) your model structure/architecture.

15 Marks **Analysis of results**

You should include quantitative (number based) and qualitative (example based) evaluations of different approaches that you have tried (on the held-out validation set).

- Quantitative evaluation includes at least measuring the cumulative error distribution or using boxplots (or other plots) to compare methods.

- Qualitative evaluation includes at least showing example images that visually demonstrate performance.
- Explicitly define any evaluation metrics and ensure they are appropriate for the task.
- Note that we are not only interested in your final prediction results, but also your explanation of why they performed well or badly. For example, you should identify failure cases and systematic bias, and provide plausible explanations for why they occur.

10 Marks **Extended analysis of robustness**

Choose one approach and analyse how robust it is to noise and additional sources of variability. For example, by adding increasing levels of Gaussian noise, or performing transformations on the images. Comment on why a chosen approach might be robust to certain types of changes in the input.

10 Marks **Accuracy of face alignment**

These marks are allocated based on the performance of one trained model of your choice. This will be evaluated on the held out test data. The test images (without points) are provided in the compressed numpy file accessible from the corresponding link in Table 2. The error on the predicted points will be calculated after submission. Marks will be awarded for average accuracy (% of images with error below a certain threshold). Note that only we have the test points!

4.2 Task 2 most important links

Contents	File name	Link
Training images and points	face_alignment_training_data.npz	link5
Validation images and points	face_alignment_validation_data.npz	link6
Test images (without points)	face_alignment_test_data.npz	link7
Colab worksheet with some useful functions	aml_task2_cv_worksheet.ipynb	link8

Table 2: Task 2 important links.

4.3 Where to start?

Face alignment is covered in week 9, lecture 18. Other lectures in weeks 8-10 are also helpful. We have included a basic Colab worksheet (see link in Table 2) illustrating how to load the data and visualise the points on the face. A visualisation of the average face and points across all training images is given in Figure 2.

5 General points on the report

- Read things! Provide references to anything you find useful. You can take figures from other works as long as you reference them appropriately.
- Diagrams, flowcharts and pictures are expected! Make sure you label them properly and refer to them from the text.

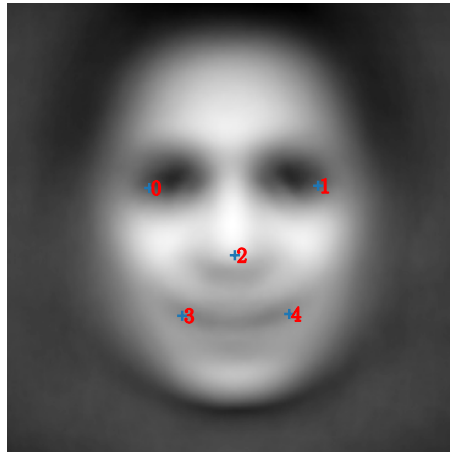


Figure 2: Illustration of the **0-indexed** (counting from 0 as you would in Python) locations of the points on the average face. For example, if we wanted to find the nose, that's index 2 so we would look up `points[2,:]`, which would give you the x and y coordinates.

- All plots should have correctly labelled axis and the font sizes must be readable in A4 page format.
- All figures (including plots) should have descriptive captions.

6 Notes on using Colab

Either you can complete this project using Google Colab or you can use your personal/lab machine. If you are using Colab, try to familiarise yourself with some of its **useful features**. To keep your saved models, preprocessed data etc. you can save it to Google Drive by following **these instructions provided by Medium**. If you refactor code into extra .py files, these should be stored in your Google drive as well, or on Box such that they are easy to load into your Colab worksheet. Alternatively, if you use GitHub to make a repository for your code, you can directly interface it with Google Colab by following **these instructions provided by Google Colab**.

7 What software functionality can I use?

- **You are not** allowed to use generative AI tools (e.g., Copilot etc.) to write your report.
- Furthermore, **you are not** allowed to use generative AI (including AI-agents) “end-to-end” to analyse data that you have uploaded, because it can be difficult - or even impossible - to evaluate the code that was used to produce the results.
- **You are** allowed to use generative AI to help you query ideas and code, but please include a statement in your report describing how you used it. Furthermore, always make sure to double-check the response. Remember that these tools have a disclaimer for a reason!

- Please see the Canvas module information page for more information about the use of generative AI: <https://canvas.sussex.ac.uk/courses/36399/pages/module-information>.
- **You are not** allowed to use library functions that have been written to directly solve the tasks you have been given, i.e. text classification and face alignment. For Task 2 specifically, **you are not** allowed to use the dlib or mediapipe face alignment tools or anything that provides similar functionality. Also, note that face detection is not required.
- **You are** free to use fundamental components and functions from libraries such as NLTK, OpenCV, numpy, scipy, scikitlearn to solve this assignment, although you don't have to. Here, fundamental components refers to things like regression / classification models and pre-processing / feature extraction steps and other basic functionality.
- In terms of tools and frameworks, **you are** allowed to use deep learning techniques such as recurrent neural networks (covered in lecture 8) and convolutional neural networks (covered in lectures 19-20) if you want to. The best package for Python-based deep learning is PyTorch. If you use such an approach you should be sure to document how you chose the architecture and loss function. A well justified and high performing deep learning approach will receive equivalently high marks as if you had built it any other way.
- **You are not** allowed to source additional labelled data for this assignment. This is because in real-world commercial projects you will typically have a finite dataset, and even if there are possibly useful public datasets available, licenses often prohibit commercial use.
- On the other hand data augmentation, which effectively synthesises additional training examples from the labelled data that you have, is highly encouraged. If you use this, please add some text or a flow-chart of this process in your report.

8 Top tips for success

- Refer to lecture slides and labs - the information is in there to complete these tasks!
- Remember Occam's razor: complexity should not be added unnecessarily. The more complicated your system the more things to explain/justify etc.
- Start with a simple achievable goal and use that as a baseline to test against. Keep track of early models/results to use as points of comparison.
- Think about things that you have learned about in other modules, e.g., Introduction to Data Science. Dimensionality reduction and clustering could be helpful. Overfitting and outliers may be an issue, and you should consider using methods to minimise this.
- For Task 2: You don't need to work at high resolution to get accurate results. Particularly when doing initial tests, resize your images to a lower resolution images. Make sure you also transform your training points so they are in the same geometry as the image (i.e., if you half the size of the image along both axes, then make sure to half the (x,y) position of the training points too). For your predicted points, make sure these are all at the same resolution as the original images.
- Even if you get stuck, at least try both tasks!